PGCP (VLSI) · Verilog HDL: from modules to a working UART · case study 2

Assignment — the divider

Report. Answer Q1 to Q6 on paper before you write any Verilog, then copy them here. The names you use in Q1 to Q3 must be the names in your code.

Name	BHUSHAN JADHAV
Roll No.	10
GitHub username	cdacbhushanjadhav-hub
Repository	https://github.com/cdacbhushanjadhav-hub/10-Bhushan-assignment
Hints used (section 10 of the sheet)	1: Q1, Q2; 3: Q5, latency; 4: Q9

Q1 Registers before states

What must the circuit remember between clock edges? One row per register, state register included. The inputs may change after the start edge.

register	bits	what it holds	why it must exist
state	2	Current FSM state: IDLE, RUN, or DONE	Remembers which control phase is active between clock edges.
A	8	Latched dividend a at start	Declared in RTL; note synthesis removes it because it is not used by the datapath.
B	8	Latched divisor b	Keeps the divisor stable throughout the operation.
R	8	Running remainder; initialized to a and reduced by B	Stores the current value being compared and subtracted.
Q	8	Internal quotient counter; initialized to 0 and incremented per subtraction	Counts successful subtractions.
q	8	Final quotient output, copied from Q when R < B	Holds the result for the external circuit.
r	8	Final remainder output, copied from R when R < B	Holds the result for the external circuit.
done	1	Completion flag	Signals completion; set to 1 in the DONE branch and cleared in IDLE.

Q2 Transfers

For each register: what is written into it and when, as register ← expression. Each distinct transfer is one control signal; name it.

state (and condition)	transfer(s)	control signal
Reset asserted	state ← IDLE; A,B,R,Q,q,r ← 0; done ← 0	rst
IDLE and start=1	A ← a; B ← b; R ← a; Q ← 0; state ← RUN	start_accept
RUN and R ≥ B	R ← R − B	subtract
RUN and R ≥ B	Q ← Q + 1	increment_q
RUN and R < B	q ← Q; r ← R; state ← DONE	finish_calculation
DONE	done ← 1; state ← IDLE	signal_done
IDLE	done ← 0	clear_done
default state	state ← IDLE; done ← 0	recover_default

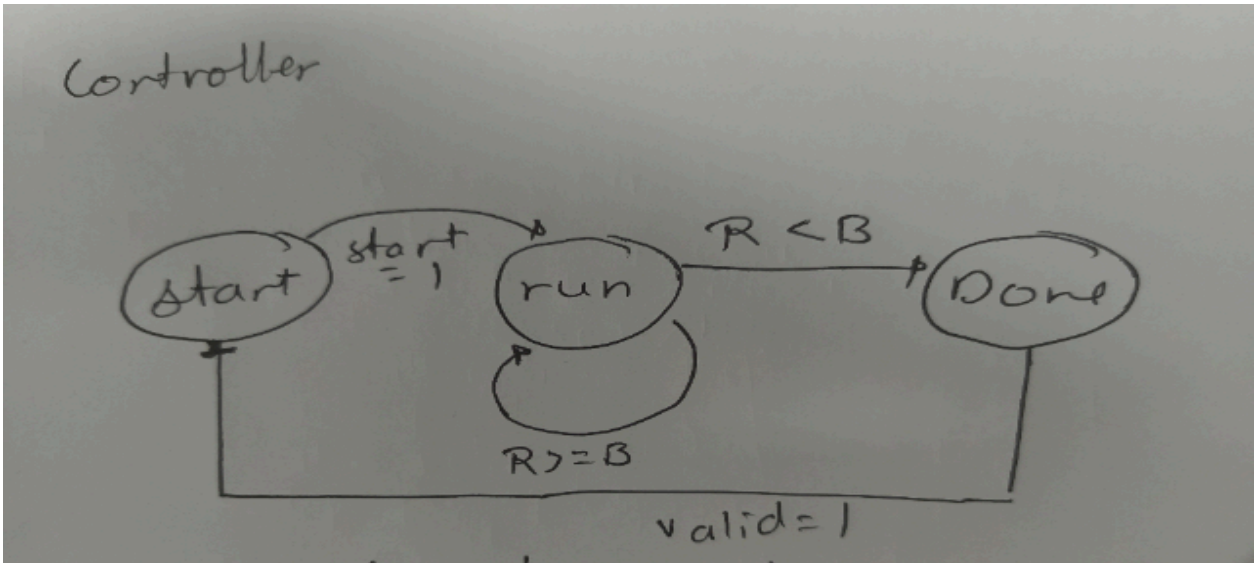
Q3 The status signal

The one bit the controller needs to decide, as an expression of registers.

name	expression	computed from (registers)
ge	(R >= B)	R and B

Q4 State diagram

Draw it with the transfers on the arcs. Paste a photo or a drawing below (Insert → Pictures), then classify every output.



output	Moore or Mealy	why
done	Moore-style registered pulse	Assigned in the DONE state branch; it is cleared in IDLE.
q	Mealy-style / condition-controlled registered output	Updated when RUN observes R < B; captures Q.
r	Mealy-style / condition-controlled registered output	Updated when RUN observes R < B; captures R.

Q5 Per-cycle tables, before any code

Edge 0 is the edge that samples start; the values are what the registers hold after the edge. Rename the register columns to yours and leave unused ones empty; stop at the first row after done and leave the remaining rows empty. A test reads the values the state arrived with; a transfer lands at the edge that leaves it.

7 ÷ 2

edge	state after	A	B	R	Q	done	what this edge did
0	RUN	7	2	7	0	0	start sampled; A←7, B←2, R←7, Q←0
1	RUN	7	2	5	1	0	R≥B: subtract 2; increment Q
2	RUN	7	2	3	2	0	R≥B: subtract 2; increment Q
3	RUN	7	2	1	3	0	R≥B: subtract 2; increment Q
4	DONE	7	2	1	3	0	R<B: q←3, r←1; enter DONE

edge	state after	A	B	R	Q	done	what this edge did
5	IDLE	7	2	1	3	1	DONE branch sets done=1; state returns to IDLE
6							
7							

5 ÷ 9

edge	state after	A	B	R	Q	done	what this edge did
0	RUN	5	9	5	0	0	start sampled; A←5, B←9, R←5, Q←0
1	DONE	5	9	5	0	0	R<B immediately: q←0, r←5; enter DONE
2	IDLE	5	9	5	0	1	DONE branch sets done=1; state returns to IDLE
3							
4							

Q6 The longest case

255 ÷ 1: how many edges from edge 0 to done, and what does that mean for a testbench that gives up after a fixed number of edges?

edges from edge 0 to done	what it means for the testbench
257 counted edges after the start-sampling edge (testbench reports 257 for 255÷1)	A fixed timeout must be greater than the worst-case completion time; the 600-edge timeout used here is sufficient.

Q7 The run

Paste the last line of tb_div_sub.v exactly as printed. Required: 0 mismatches -> PASS; done after q+1 edges on 8 of 8.

```
assig_q1_tb: 8 divisions, 0 mismatches -> PASS; done after q+1 edges on 0 of 8
```

Q8 Flop count

Predict before you synthesize, from Q1 and the encoding you assume; then paste the Register as Flip Flop line; then reconcile.

Predicted (from Q1), with the state encoding assumed	51 bits: A,B,R,Q,q,r are six 8-bit registers (48), state is 2 bits, done is 1 bit. Assumes the RTL's 2-bit state register.
Reported (Register as Flip Flop line)	43 flip-flops (Vivado cell usage: FDCE = 43; utilization: Slice Registers = 43).
Encoding Vivado chose (Synth 8-line)	Not explicitly reported in the supplied synthesis log. RTL constants are IDLE=00, RUN=01, DONE=10.
Why they agree or differ (one sentence)	Vivado removed the unused 8-bit A_reg, so 51 predicted bits − 8 unused bits = 43 synthesized flip-flops.

Q9 Two pairs from the testbench

Which decision in your code does each pair exercise, and what would the output have been with the other choice?

pair	decision in your code it tests	output with the other choice
200 ÷ 200 (a = b)	Exercises the R ≥ B branch when R equals B; it must subtract once and increment Q.	If it took the R < B choice, it would stop immediately with q=0, r=200 instead of q=1, r=0.
255 ÷ 1 (b = 1)	Exercises the maximum quotient and longest repeated-subtraction run: 255 successful subtractions.	If the circuit stopped on R = B (using > instead of ≥), it would stop at R=1 with q=254, r=1 instead of q=255, r=0.

Before you push: this file saved as report.docx in your repository folder, the diagram inside it, and the checklist in section 8 of assignment.md read once against your code.